

Parking Space Detection and Classification using Classical Image Processing Techniques

Rice, Michael
MSc Artificial Intelligence
University of Galway
Galway, Ireland
m.rice11@universityofgalway.ie

Abstract – This document aims to display methods used to develop and test a parking space detection and classification system using solely classical image processing techniques. It aims to suit resource-constrained embedded systems applications better than a Deep Neural Network (DNN) approach. This algorithm will use core image processing tools such as Python and its available libraries (OpenCV, NumPy etc.)

Keywords – Python, OpenCV, NumPy, Gaussian Blur, Morphological Operations, Harris Corner Detection, Filtering, Bounding Boxes, Region of Interest, Greyscale, RGB, Contours, Canny Edge Detection, Deep Neural Network, Deep Learning

I. INTRODUCTION

Parking space occupancy detection is a tool that plays a significant role in urban management systems. Legacy approaches to this task have traditionally used either physical (Ultrasonic/Magnetic etc.) sensors placed on the ceiling or floor of a parking space or a ticketing system to track the number of cars in a car park. However, due to recent developments, the prominence of lightweight image processing techniques, as well as deep learning approaches, have emerged as potential alternatives.

To approach this assignment, the utility (to this task) of all the relevant techniques learned from previous assignments was analyzed. These ranged from basic pixel-level operations to much more complex concepts such as corner detection or basic feature extraction techniques. Each of their potential values for completing sub-tasks were evaluated. By examining these classical techniques, it provided a well-rounded understanding of the realm of image processing.

This paper will give an overview of the steps necessary to implement the algorithm this analysis produced, from the basic building blocks down to the function of each tunable parameter.

II. METHODOLOGY

A. Base Algorithm

To tackle this task, a general algorithmic pipeline was initially constructed to tackle all images, with alterations or additions coming into play for the later sets.

It incorporated several key classical image processing techniques such as corner detection, color space conversions, blurring and more.

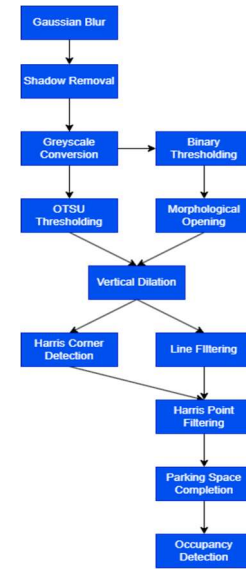


Figure 1 : Base Algorithm Flowchart

B. Algorithm Operation Blocks and Parameter Influence

To explain the base algorithm, the function of each algorithmic building block, as well as their parameters, will be discussed (not in order of application), following a brief pipeline overview.

0) Pipeline Overview

A brief overview of the approach taken to this assignment is as follows: To detect the parking space lines, morphological operations were used to isolate the horizontal and vertical lines before closing the parking space boxes by finding the y coordinates with the highest quantity of nearby Harris Corner detections and drawing lines at those coordinates. To then detect whether the spaces were occupied or not, the Harris Corners in the centers of the detected parking spaces were counted and if the count was above a threshold the space was classified as occupied.

1) Gaussian Blur

Gaussian Blur was used as the first step of this pipeline to remove high frequency noise that could potentially harm line detections. Gaussian blur uses a weighted average system where values closer to the center of the kernel will contribute more to the average [1], which helps to preserve edges.

A 3x3 kernel was used for a lower amount of blurring compared to higher values, to again preserve edges as best as

possible for future detections while removing small amounts of noise.

2) Shadow Removal

As the images provided were exclusively outdoors, shadow removal was necessary to remove the high contrast edges they produce. A morphological Top Hat operation was used to do so, which is the difference between an image and the morphologically opened version of the image.[2]

A large kernel of 15x15 was used to retain the large bright areas outside of shadowed regions.

3) Greyscale Conversion

A greyscale conversion was necessary for input to the two thresholding functions.

4) Thresholding

Thresholding is then used to convert all values above a base level to the max value for an 8-bit integer representation, 255. This helps to highlight edges and lines in the image. Both Otsu (automatic) and Binary (pixel values of above 50) thresholding were used in this pipeline for differing tasks,

5) Morphological Dilation

Another morphological operation was subsequently used, dilation, to extend vertical lines detected in the images [2]. A kernel value of (1,10) was used to emphasize this vertical plane.

6) Morphological Opening

Opening, again a morphological operation, is a combination of two lower-level ones, erosion followed by dilation. [2] This combination serves to remove small noise in the image, often referred to as ‘salt-and-pepper’ noise.

When using opening for this task, I chose two main kernel types, one x-coordinate dominated, and one y-coordinate dominated. These were necessary to isolate both only the horizontal and vertical lines in each image. The resulting images could then be combined to remove all but the parking grid lines.

If the x value for the x kernel and the reverse for the y kernel were enlarged it would detect larger lines in the respective plane, and in turn, if the values were decreased, smaller lines would remain.

7) Harris Corner Detection

Corner Detection is an effective method for discerning changes in intensities across two planes. This technique was useful for parking space detection for detecting the intersection points of a parking grid. [3]

For this implementation, the parameter values were as follows:

- threshold: 0.25, controls sensitivity of corner detection, the lower the more sensitive.

- blockSize: 2, size of considered neighborhood, the larger the bigger the neighborhood.
- ksize: 3, size of Sobel Kernel used for edge detections, the larger the bigger the kernel and the stronger the edge detection applied.
- k: 0.04, Harris equation free parameter.

8) Line Filtering

To distinguish between lines required for the parking spaces and those who were not, a filtering approach was taken in which detected lines were filtered out depending on if they were above a length threshold and below a width threshold.

This was done by fitting a bounding box to all the detected contours in an input image and processing the contour based on the dimensions of the bounding box.

For example, the initial set of images had values between 175 and 350 for the vertical line threshold and either 5 or 50 for the width threshold, image dependent.

9) Harris Point Filtering

Following the filtering of the lines, the previously detected Harris Corner points were also filtered so that only those who overlapped with the detected lines were kept.

10) Parking Space Detection

Taking these overlapping Harris Points, the outlines of the parking spaces were found using a histogram of the points’ y-coordinates. By quantizing their coordinates, the bins with the most values in them then identify the locations where parking spaces end, due to corners being present at the end of their outline.

A function argument ‘num_lines’ was used to dictate the number of these bins to select to have lines drawn at their coordinate in the image.

With all the necessary lines being in place, parking spaces could finally be identified. This was again done using contours, as well as aspect-ratio and area-based filtering. Parameters into the ‘find_parking_spaces’ function controlled the required parking space aspect ratio for a given image as well as the amount over the mean area of the images the area of a current threshold could be before being discarded.

11) Parking Space Classification

Finally, with the required parking spaces detected, they could be classified as occupied or not. This was done using the previously detected Harris Corner points.

First, each parking space was cropped to remove the area near the edges. Secondly, the number of detected corner points in this cropped region was counted and if it was above a threshold, the space was deemed occupied.

C. Algorithmic Alterations for Medium Set

Aside from several tunable parameters being changed, there were a few larger system changes to get the best performance out of the medium and hard sets.

0) Perspective Transform for Image 2

Due to the nature of Image 2 in the medium set, a new technique would be required to get correct detections, a perspective transform. This helps to view an image from another angle, making tasks such as straight line detection easier[4]. For this precise purpose I used two distinct transformative steps to alter the image.

Firstly, as the image displayed some warping or distortion on the right side, I wished to drag the top and bottom right-hand corners of the parking lot upwards. This helped to make the lot more linear and reduced the distortion on the right-hand side.

Secondly, to gain a better vantage point of the lines I used a Bird's Eye transformation step. This changes the image so that we appear as if we are viewing the image from above. This final step helped greatly to get clean line detections.

1) Parking Space Detections

In comparison to the easy set of images, the approach for finding the parking spaces of the medium set was slightly simpler. Instead of using the perimeter length of detected contours to approximate appropriate polygons, a bounding rectangle was simply fitted to each contour and the aspect ratio of this box was used to compare to the desired ratio.

D. Algorithmic Alterations for Hard Set

As the hard set, by design, was the most difficult to process, the original pipeline had to be modified with the changes below to work as intended.

0) Image Rotation

Firstly, the first and last images in the set were images taken of the parking lots from an angle, a characteristic I wished to amend to help with vertical line detections.

To do this cv2 defined Affine Transformations[5] of 18 and 2.5 degrees respectively were used.

1) HSV Colour Masking

Another image specific change needed for the hard set was a color mask using the HSV (Hue Saturation Value) color space[6]. The use of this color space enabled the selection of specific colors and by extension, masking them in or out of the image.

The reason this technique was required lies in the fact that blue parking lines make up some of the lines in the second hard image, meaning some form of color-based technique

was the only way the detection of these lines could be suitably attained for my pipeline.

2) Gap Based Line Filtering

As the resolution of these images were much greater than their predecessors, the separation of appropriate lines from those which were unwanted became more difficult. For this purpose, a new parameter named *gap_threshold*, was introduced to the line filtering process.

This parameter made it impossible for two lines to be detected within a certain distance in the x plane, as if a second line was detected too close to an already present line, this second line was removed from the coordinates list.

3) Width & Height Filtering

A common change introduced for all images in the hard set except the first one, was to change from aspect ratio to explicit height and width for detecting the parking space contours.

This was done to be more precise with the contour filtering process.

4) Y-Coordinate Based Filtering

Alongside the height and width filtering, coordinate filtering techniques were introduced based on the y coordinates of detected contours, tailored for purpose.

This performed the function of allowing different heights and widths of desired parking space to be applied to different sections of the pre-detected lines.

Furthermore, for the second image, the boxes detected in certain regions were clipped should they extend past a certain height to prevent them extending slightly over sections of road.

III. RESULTS

A. Accuracy Analysis

Please find the final images attached in the Appendix. [0]

0) Easy Set

Image	Parking Spaces	Detections	True Pos	True Neg	Errors	Accuracy
1	26	24	16	8	2	92.3%
2	24	24	17	7	0	100%
3	14	12	0	12	2	85.7%
4	14	14	4	10	0	100%
Total	78	74	37	37	4	94.9%

Table 1: Easy Set Accuracy Results

1) Medium Set

Image	Parking Spaces	Detections	True Pos	True Neg	Errors	Accuracy
1	39	39	10	26	3	92.3%
2	34	34	15	17	2	94.1%
3	22	22	11	11	0	100%
Total	95	95	36	54	5	95.5%

Table 2: Medium Set Accuracy Results

2) Hard Set

Image	Parking Spaces	Detections	True Pos	True Neg	Errors	Accuracy
1	8	7	6	0	2	75%
2	194	165	17	131	46	76.3%
3	35	32	4	22	9	74.3%
Total	238	204	27	153	58	74.6%

Table 3: Hard Set Accuracy Results

3) Reasons For Inaccuracies

a) Easy

In the easy set, there was one main reason for detection errors, being the presence of broken or obstructed parking lines. These factors meant that the lines were not of sufficient length to avoid being filtered out by the line filtering step in the algorithm. If the threshold had been lowered, a sizeable number of noisy lines would have been introduced.

b) Medium

For the medium set, two sources of error affected the detections of the parking spaces. Firstly, the distortion in the second image makes perfect detections difficult due to the curvature of the parking space lines, as well as the off-center nature of the cars.

Secondly, in the first image from this set, the right-hand line of the top right hand parking space merges with the line from the grass section, rendering it undetectable with this algorithm. Also, three parking spaces are detected to the right of the grassy section where they do not exist. This was due to lines from other detections, as well as the line from the right side of the grassy section crossing over to create rectangles of the desired aspect ratio.

c) Hard

For the final set, the hard set, there were more factors that affected the parking space's detections in a negative way.

As before, for the first image in the set, broken or obstructed lines affected the accuracy of the predictions.

For the second image a variety of factors could potentially have damaged the overall accuracy including: number of lines, Harris point detection sensitivity, overlapping lines and many more. It is hard to pinpoint the exact reasons for the inaccuracies in this image due to the sheer number of parking spaces.

Finally, for the third image, the largest error source came from the adaptability of the algorithm to the image. As the two rows of parking spaces were misaligned, the algorithm struggled to differentiate between the two sets of vertical

parking lines, causing less accurate detections. As well as this, the overall texture of some of the parking spaces caused some difficulties.

B. Code Complexity Analysis

Using Big O analysis [5], I computed the time complexity of each of the building blocks of the pipeline.

a) Easy / Medium Sets

*** assuming complexity on a one image basis***

Function	Big O Complexity
harris corner detection	$O(n)$
filter lines	$O(n * \log n)$
completing boxes	$O(n)$
find parking spaces easy / medium	$O(n * \log n)$
fix camera distortion	$O(n)$

b) Hard Set

Function	Big O Complexity
rotate h0 / h3	$O(n)$
blue and white color masking	$O(n)$
filter lines h0 / h1 / h3	$O(n \cdot \log n)$
completing boxes h0 / h1 / h3	$O(n)$
harris corner detection hard	$O(n)$
find parking spaces h0 / h1 / h3	$O(n \cdot \log n)$

n = number of pixels

Overall Pipeline Complexity: $O(n \cdot \log(n))$

By this complexity analysis, it is evident that for this pipeline complexity is most affected by the number of lines as well as the number of parking spaces.

C. Time Efficiency Analysis

As an impact of the code complexity analysis, the higher the image resolution, the longer each image took to process. Below are the durations taken, set by set.

0) Set Comparison

a) Easy

For the easy set, image sizes were lower than for the other two sets, as well as the pipeline being simpler overall. Therefore, without plotting, the four images in this set took a combined 0.3 seconds to process, for an average of **75ms** per image.

The average image size in this set was **725x460**, or **0.36 MP**. This means that the rate of processing for this set was **4.84 MP/s**.

b) Medium

The image sizes in the medium set took an upwards jump to an average size of **1.27MP**, with average dimensions of **1407x770 pixels**. This was almost a four times increase from the easy set.

In response, the three images took 2 seconds to process, giving a processing rate of **1.903 MP/s**, representing a processing rate of less than half the easy set.

c) Hard

Finally, for the hard set the average image dimensions were **2927x1684**, giving an average image size of **5.8MP** (heavily weighted towards the final two images), this time being almost a five times increase from the previous set.

The processing time for the images totaled 17.9s, with 87% of the time coming from image two. This gives a processing rate of **0.972 MP/s**.

If image two from this set had a similar processing time to the other image of the same resolution, a much higher rate of 3.51 MP/s could have been achieved.

Without the impact of image two from the hard set, the overall processing rate would be 4.55 MP/s, instead it falls to 1.09 MP/s.

IV. DISCUSSION

A. Faults

0) High Corner & Line Density Sensitivity

As seen in the Time Complexity analysis, the time required to process certain images can be extensive. Using this algorithm, one image from the hard set took over 15 seconds to process.

This is due to the sheer number of lines/corners, and by proxy parking spaces, detected in the image. There were 191 parking spaces in this image, almost five times as many as the next most. Due to this, when corner/line intersections were being calculated, a much more extensive search for overlaps was required.

This length of processing is unreasonable for the application, therefore if the algorithm were to be improved, fixing this would have to be a high priority.

1) Large Number of Changeable Parameters

Since this pipeline had to be implemented in ten distinct parking scenarios, a sizeable number of parameters had to be introduced to control the system's behavior.

While this gives the system a degree of overall flexibility, it requires some unique valued parameters for each image, reducing its ability to automatically adapt to differing scenarios.

B. Positive Aspects

0) Strong Performance

As is evident from the accuracy analysis section, this algorithm performs well when the conditions are favorable, and its parameters are tuned for the application. In the easy and medium sets an average performance of over 95% was achieved as well as an overall performance of about 88%.

1) Interpretability

Due to the nature of the building blocks combined to build this pipeline, the results are highly interpretable, as the results from each block can be examined in a sequential manner. This allows for easier debugging and fine-tuning.

2) Required Resources

As the techniques deployed in this algorithm are not based on machine learning, training data is not required, which can greatly reduce the cost and increase the ease of an implementation in a real-world environment.

C. Embedded System Discussion

Using the available compute, images can be processed at a maximum rate of **13.33 fps**, for live video applications (using the lowest single image execution time).

However, for parking detection applications, if images are at a maximum of 720x720, resulting in 518,400 pixel operations per block, a quantity that can be handled in a number of ms by various embedded processors such as the Teensy 4.0.[7], making this algorithm suitable for embedded systems, at least in simplistic scenarios.

Even though numerous blocks are required to get the result, should an image be put through the pipeline every second, for power conservation reasons, this would still be a real-world applicable solution, as the temporal granularity necessary for accurate systems is relatively low, as it is not critical for counts to be updated more often than every second.

However, with increased resolution operation counts quickly become infeasible even for the 600 MHz max of the Teensy.[7]

D. Comparison with DNNs

The top end of the operation count range per image for this algorithm, (somewhere in the range of 5-10 million operations for 720x720 images) is still an achievable quantity for microcontrollers. However, even lightweight DNNs such as shallow CNNs can require tens or hundreds of millions of operations per forward pass, without pruning or quantization. For example, EfficientNet B0, a very light implementation, reaches 390 million FLOPs (floating point operations) per forward pass.[8]

However, algorithms implemented with these classical IP techniques are less adaptable to changes in input data than DNNs (which themselves are subject to the quality of training data), making them generalize better than this algorithm to unseen scenarios.

Finally, DNNs typically require some form of hardware acceleration for efficient inference[9]. Implementing such systems without GPUs/NPUs etc can drastically affect performance, power consumption and more. However, the expense of such hardware can impact such projects due to financial or spatial constraints.

E. Downsampling

In the brief for this assignment, down sampling images was mentioned as a potential improvement of an implementation.

As seen in the complexity analysis section, the overall complexity of the system is controlled by the number of pixels in the images. Further, as seen in the easy set, it is possible to get perfect detections on images of lower resolutions. However, I do not believe that performing such down sampling would be an easy solution for increases in time complexity. This could be due to loss of detail / softening of edges / corners in the resulting images.

Finally, due to the number of parameters necessary for tuning in this algorithm, down sampling would result in a restart of this optimization process, increasing the time spent on the solution overall.

F. Future Work / Improvements

If more time were available to improve this work, the following are the changes that I would implement.

0) Code Tidying

There are several functions of similar names in my helper functions section that perform the same task for different image(s). Combining these function groups into singular functions would be useful for readability, however, it would not be vital to implement.

1) Downsampling Investigation

As mentioned previously, restarting the overall process with down sampled images would prove time consuming, however if results were of the utmost importance, this would be a potential avenue to improve one of them, the temporal performance.

2) Corner Detection Length of Processing

As mentioned twice previously, this algorithm struggles for efficiency in the presence of a large quantity of Harris Corner points and/or parking lines.

To remedy this issue, three approaches could be taken to solve it.

- First, attempt to find a better balance between corner detection sensitivity and temporal performance. This could prove time-consuming and/or impossible.
- Secondly, if the above does not work, consider another corner detection algorithm such as FAST or Shi-Tomasi.
- Finally, if neither one nor both of the above work, a system of implementing ROIs over grids of parking spaces could also prove worthwhile to narrow the search space.

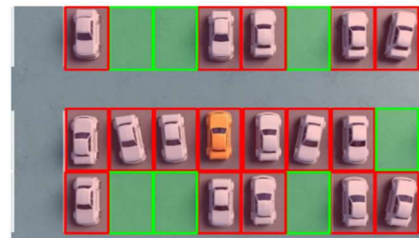
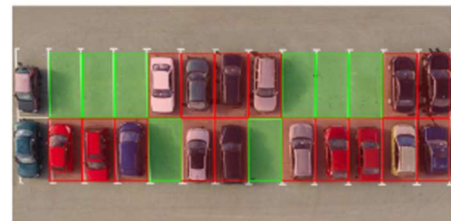
V. CONCLUSION

Finally, to add my own opinion to this paper, I think that completing this task was extremely valuable in terms of understanding the base concepts of Image Processing, in an embedded systems context, from the base up.

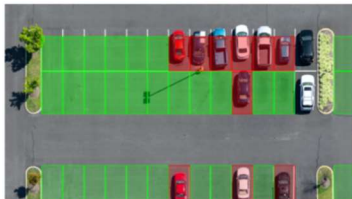
To conclude, I believe that my approach to this task was intuitive, largely effective and, for the most part, efficient. This was by no means an easy project, however, the learnings available were plentiful and are something I will take forward into future roles, research and more.

VI. APPENDIX

EASY RESULTS



MEDIUM RESULTS



HARD RESULTS



REFERENCES

- [1] "OpenCV: Smoothing Images." Accessed: Nov. 26, 2024. [Online]. Available: https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html
- [2] "OpenCV: Morphological Transformations." Accessed: Nov. 26, 2024. [Online]. Available: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html
- [3] "OpenCV: Harris Corner Detection." Accessed: Nov. 26, 2024. [Online]. Available: https://docs.opencv.org/4.x/dc/d0d/tutorial_py_features_harris.html
- [4] "OpenCV: Geometric Image Transformations." Accessed: Nov. 26, 2024. [Online]. Available: https://docs.opencv.org/4.x/da/d54/group_imgproc_transform.html
- [5] "OpenCV: Affine Transformations." Accessed: Nov. 27, 2024. [Online]. Available: https://docs.opencv.org/4.x/d4/d61/tutorial_warp_affine.html
- [6] "Color Spaces in OpenCV | Python," GeeksforGeeks. Accessed: Nov. 27, 2024. [Online]. Available: <https://www.geeksforgeeks.org/color-spaces-in-opencv-python/>
- [7] By, "New Teensy 4.0 Blows Away Benchmarks, Implements Self-Recovery, Returns To Smaller Form," Hackaday. Accessed: Nov. 27, 2024. [Online]. Available: <https://hackaday.com/2019/08/07/new-teensy-4-0-blows-away-benchmarks-implements-self-recovery-returns-to-smaller-form/>
- [8] T. A. Team, "EfficientNet — An Elegant, Powerful CNN. | Towards AI." Accessed: Nov. 29, 2024. [Online]. Available: <https://towardsai.net/p/l/efficientnet%e2%80%8a-%e2%80%8aan-elegant-powerful-cnn>
- [9] G. Sciangula, F. Restuccia, A. Biondi, and G. Buttazzo, "Hardware Acceleration of Deep Neural Networks for Autonomous Driving on FPGA-based SoC," in *2022 25th Euromicro Conference on Digital System Design (DSD)*, Aug. 2022, pp. 406–414. doi: 10.1109/DSD57027.2022.00061.